Tutorial-2

Theory

Theta Notation (Θ-Notation):
Θ(g(n)) = {f(n): there exist positive constants c1, c2 and n0 such that 0 ≤ c1 * g(n) ≤ f(n) ≤ c2 * g(n) for all n ≥ n0}

Big-O Notation (O-notation):
O(g(n)) = { f(n): there exist positive constants c and n0 such that 0 ≤ f(n) ≤ cg(n) for all n ≥ n0 }

Omega Notation (Ω-Notation):
Ω(g(n)) = { f(n): there exist positive constants c and n0 such that 0 ≤ cg(n) ≤ f(n) for all n ≥ n0 }

Master Method

The master theorem concerns recurrence relations of the form:

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n) \quad \text{where } a \geq 1,\, b > 1$$

In the application to the analysis of a recursive algorithm, the constants and function take on the following significance:

- n is the size of the problem.
- a is the number of subproblems in the recursion.
- n/b is the size of each subproblem. (Here it is assumed that all subproblems are essentially the same size.)
- $f(n)$ is the cost of the work done outside the recursive calls, which includes the cost of dividing the problem and the cost of merging the solutions to the subproblems.

Case 1

Generic form

If $f(n) = \Theta\left(n^c\right)$ where $c < \log_b a$ (using Big O notation) then:

$$T(n) = \Theta\left(n^{\log_b a}\right)$$

Case 2

Generic form

If it is true, for some constant $k \geq 0$, that:

$$f(n) = \Theta\left(n^c \log^k n\right) \text{ where } c = \log_b a$$

then:

$$T(n) = \Theta\left(n^c \log^{k+1} n\right)$$

Case 3

Generic form

If it is true that:

$$f(n) = \Theta\left(n^c\right) \text{ where } c > \log_b a$$

then:

$$T(n) = \Theta\left(f(n)\right)$$

Solved Example using substitution method:

(I) Substitution Method

(1) $T(n) = \begin{cases} T(n-1)+1, & n>1 \\ 1, & n \leq 1 \end{cases}$

$\rightarrow T(n) = T(n-1) + 1$

$= T(n-2) + 1 + 1$

$= T(n-3) + 1 + 1 + 1$

$\vdots$

after K times

$= T(n-K) + K \quad \rightarrow \quad n-K = 1 \rightarrow K = n-1$

$= T(1) + n - 1$

$= 1 + n - 1 = n$

$\boxed{T(n) = \Theta(n)}$

Solved Example using Master Theorem

Q. eg 1. $T(n) = 8T(n/2) + n^2$

$f(n)$	$n^{\log_b a}$
n^2	$n^{\log_2 8} = n^3 \Rightarrow n^2 < n^3$

bigger by polynomial

Case I of Master's theorem

Final answer

$T(n) = \Theta(n^3)$

Solved Example using recursion tree:

(II) Recursive tree method

$$T(n) = \begin{cases} 8T(n/2) + n^2 & , n > 1 \\ 1 & , n \leq 1 \end{cases}$$

# of subproblems		excess cost
1	$T(n)$	n^2
8	$T(n/2)$ ------ $T(n/2)$	$+ \; 8\left(\frac{n}{2}\right)^2$
8^2		$+ \; 8^2\left(\frac{n}{2^2}\right)^2$
⋮		⋮
	$T\left(\frac{n}{2^{k-1}}\right)$	$+$
8^k	$T\left(\frac{n}{2^k}\right)$ ------ $T\left(\frac{n}{2^k}\right)$	$8^k\left(\frac{n}{2^{k-1}}\right)^2$

$$T(n) = 8^k T\left(\frac{n}{2^k}\right) + n^2(2^k - 1)$$

$$= 8^{\log n} \cdot 1 + n^2(n-1)$$

$$\boxed{T(n) = 2n^3 - n^2 = \Theta(n^3)}$$

$$n^2 + 2n^2 + 2^2 n^2 + \cdots \Rightarrow n^2\left[1 + 2 + 2^2 + \cdots 2^{k-1}\right]$$

$$= 2^k - 1$$

Spiral

Evaluative Questions

1. Show that if $f(n) = O(s(n))$ and $g(n) = O(r(n))$, then $f(n) + g(n) = O(s(n) + r(n))$.[5 points]

2. Recurrences [2X3 points]

Give asymptotic upper and lower bounds for T (n) in each of the following recurrences. Assume that T (n) is constant for $n \leq 3$. Make your bounds as tight as possible, and justify your answers.

(a) $T(n) = 3T(n/5) + (\lg n)^2$ (solve by master theorem)

(b) $T(n) = T(n/2) + 2^n$ (solve by method of substitution)

(c) $T(n) = 7T(n/2) + n^3$ (solve by recursion tree)

Practice Questions

A)Solve the recurrence relation:

1. $T(n)=2T(n-1)+3$

   With $T(0) = 1$.

2. $T(n) = T(n/2) + T(n/4) + n$

3. $T(n) = T(n/2) + 1$

   With $T(1) = 1$.

4. $T(n) = 2T(n/2) + n$

5. $T(n) = 2T(n/2) + \log n$

6. $T(n) = T(n-1) + T(n-2) + 1$

   With $T(0) = T(1) = 1$

7. $T(n) = 2T(n-1) + 1$

   With $T(1) = 1$

B)Consider two algorithms, Algorithm A and Algorithm B, both designed to solve the same problem. The time complexity of Algorithm A is $O(n^2)$, where 'n' is the size of the input, and the time complexity of Algorithm B is $O(2^n)$.

1) Compare the time complexities of Algorithm A and Algorithm B in terms of efficiency as the input size 'n' grows.

2) If the input size 'n' is relatively small, which algorithm would you expect to perform better in terms of runtime?

3) In what scenarios or types of problems might Algorithm A be more suitable despite its quadratic time complexity?